

Implementation Plan: Shah Sports E-Commerce Platform

Overview

This implementation plan converts the existing multi-vendor e-commerce platform into a single-vendor sports equipment store. Tasks are organized to build foundational components first, then layer on features incrementally. Each task builds on previous work to ensure no orphaned code.

Tasks

- ☐
 - 1. Database Schema Updates - Core Tables
 - ☐ 1.1 Update users migration to remove vendor types and simplify to admin/customer
 - Modify user_type enum to only allow 'admin' and 'customer'
 - Add phone field to users table
 - Remove vendor-related fields
 - *Requirements: 1.1, 1.2*
 - ☐ 1.2 Update categories migration with slug, SEO fields, and improved hierarchy
 - Add slug, description, image, sort_order, is_active fields
 - Add meta_title, meta_description for SEO
 - Update parent reference to use proper foreign key
 - *Requirements: 2.1, 2.2, 2.3*
 - ☐ 1.3 Update brands migration - remove shop reference
 - Remove shop_id foreign key
 - Add slug, sort_order, is_active fields
 - *Requirements: 4.1, 4.2*
 - ☐ 1.4 Update product_models migration - add brand relationship
 - Remove shop_id, add brand_id foreign key
 - Add timestamps
 - *Requirements: 4.3*
 - ☐ 1.5 Create shipping_classes migration
 - Create table with name, slug, description
 - *Requirements: 6.3*
- ☐

2. Database Schema Updates - Products and Variations

- ☐ 2.1 Update products migration - remove vendor references, add new fields
 - Remove vendor_shop_id, variation_id
 - Add shipping_class_id, compare_price, cost_price, low_stock_threshold
 - Add meta_title, meta_description, meta_keywords
 - Update status enum
 - *Requirements: 3.1, 3.5, 3.6, 3.7*
- ☐ 2.2 Create product_variations migration
 - Create table with product_id, sku, price, quantity, is_default
 - *Requirements: 3.2, 3.3*
- ☐ 2.3 Create variation_values migration
 - Create pivot table linking product_variations to variation_options
 - *Requirements: 3.2*
- ☐ 2.4 Update product_images migration if needed
 - Ensure is_primary field exists
 - *Requirements: 3.4*

• ☐

3. Database Schema Updates - Orders and Payments

- ☐ 3.1 Update orders migration - add POS support and remove vendor references
 - Add order_number, order_type, coupon_id
 - Add subtotal, discount_amount, tax_amount
 - Add customer_name, customer_email, customer_phone for POS
 - Add shipping_method enum with shah_sports_team and pathao_courier
 - Update status enum
 - *Requirements: 7.1, 7.2, 7.4, 8.3, 8.4*
- ☐ 3.2 Update order_items migration - remove vendor reference, add variation support
 - Remove vendor_shop_id
 - Add product_variation_id, product_name, variation_details JSON
 - Add unit_price, total_price
 - *Requirements: 7.1*
- ☐ 3.3 Update payments migration - add new payment methods
 - Update payment_method enum to include ssl_commerz, bkash, nagad, manual
 - Add gateway_response JSON, paid_at timestamp
 - *Requirements: 17.1, 17.2, 17.4*
- ☐ 3.4 Create invoices migration
 - Create table with order_id, invoice_number, dates, amounts, pdf_path
 - *Requirements: 18.1, 18.2, 18.5*

• ☐

4. Database Schema Updates - Promotions and Coupons

- ☐ 4.1 Update promotions migration - add targeting and scheduling

- Add promotion_type enum, applies_to, apply_level
 - Add starts_at, ends_at timestamps, priority
 - *Requirements: 9.1, 9.2, 9.4, 9.5*
- ☐ 4.2 Create promotion_brands and promotion_categories pivot tables
 - *Requirements: 9.5*
- ☐ 4.3 Create coupons migration
 - Create table with code, discount_type, discount_value
 - Add applies_to, usage_limit, once_per_customer
 - Add starts_at, expires_at
 - *Requirements: 10.1, 10.2, 10.3, 10.5*
- ☐ 4.4 Create coupon pivot tables (coupon_products, coupon_brands, coupon_categories)
 - *Requirements: 10.4*
- ☐ 4.5 Create coupon_usages migration
 - Track coupon_id, user_id, order_id, customer_email, discount_applied
 - *Requirements: 10.6*

• ☐

5. Database Schema Updates - Shipping

- ☐ 5.1 Update shipping_rates migration
 - Add name, shipping_class_id
 - Update method enum to shah_sports_team and pathao_courier
 - Add base_cost, is_active
 - *Requirements: 6.1, 6.2*
- ☐ 5.2 Review and update weight_cost_rules and weight_cost_rule_items
 - Ensure proper relationship with shipping_rates
 - *Requirements: 6.4*

• ☐

6. Database Schema Updates - Inventory and Returns

- ☐ 6.1 Create inventory_logs migration
 - Track product_id, product_variation_id, quantity changes
 - Add reason enum, reference_type, reference_id, notes
 - *Requirements: 5.5*
- ☐ 6.2 Update returns migration - fix foreign key and add fields
 - Fix order_item_id foreign key (currently points to orders)
 - Add quantity, reason enum, reason_details
 - *Requirements: 12.4, 12.5*
- ☐ 6.3 Update refunds migration
 - Add return_id, refund_type, processed_at
 - Update refund_method enum
 - *Requirements: 12.3, 12.4*

- ☐

7. Database Schema Updates - Reviews, Campaigns, CMS

- ☐ 7.1 Update reviews migration - remove shop reference
 - Remove shop_id foreign key
 - Add title, helpful_count, admin_response
 - *Requirements: 13.1, 13.3, 13.5*
- ☐ 7.2 Create review_helpful pivot table
 - Track user_id, review_id for helpful votes
 - *Requirements: 13.5*
- ☐ 7.3 Create campaigns migration
 - Add name, subject, content, campaign_type, target_type
 - Add scheduling and tracking fields
 - *Requirements: 11.1, 11.2, 11.5*
- ☐ 7.4 Create campaign_recipients migration
 - Track campaign_id, user_id, email, status, timestamps
 - *Requirements: 11.3*
- ☐ 7.5 Update store_policies migration - remove shop reference
 - Remove user_id, shop_id, is_default
 - Add title, slug
 - Update policy_type enum
 - *Requirements: 14.1*
- ☐ 7.6 Create cms_pages migration
 - Add title, slug, content, meta fields, is_active
 - *Requirements: 14.2, 14.4*
- ☐ 7.7 Update promotional_banners migration
 - Rename to banners, add subtitle, position enum
 - Add starts_at, ends_at, is_active
 - *Requirements: 15.1, 15.2, 15.3*

- ☐

8. Checkpoint - Database Migrations

- Run all migrations and verify schema
- Ensure all tests pass, ask the user if questions arise.

- ☐

9. Update Eloquent Models - Core

- ☐ 9.1 Update User model
 - Update fillable, casts, user_type accessor
 - Add relationships: profile, addresses, orders, reviews, wishlists
 - Add isAdmin(), isCustomer() helper methods
 - *Requirements: 1.1, 1.4*
- ☐ 9.2 Update Category model

- Add slug generation, parent/children relationships
 - Add products relationship, scopeActive
 - *Requirements: 2.1, 2.2, 2.5*
 - ☐ 9.3 Update Brand model
 - Remove shop relationship
 - Add slug generation, products and models relationships
 - *Requirements: 4.1, 4.2*
 - ☐ 9.4 Update ProductModel model
 - Add brand relationship, remove shop
 - *Requirements: 4.3*
 - ☐ 9.5 Create ShippingClass model
 - Add products relationship
 - *Requirements: 6.3*
- ☐
- 10. Update Eloquent Models - Products
 - ☐ 10.1 Update Product model
 - Remove vendor relationships
 - Add shippingClass, variations relationships
 - Add scopeActive, scopeFeatured, scopeTrending
 - Add getStockStatusAttribute accessor
 - *Requirements: 3.1, 3.2, 3.6, 5.6*
 - ☐ 10.2 Create ProductVariation model
 - Add product, variationValues relationships
 - Add stock management methods
 - *Requirements: 3.2, 3.3*
 - ☐ 10.3 Create VariationValue model
 - Add productVariation, variationOption relationships
 - *Requirements: 3.2*
 - []* 10.4 Write property test for product variation inventory independence
 - **Property 7: Product Variation Inventory Independence**
 - **Validates: Requirements 3.2, 3.3**
- ☐
- 11. Update Eloquent Models - Orders
 - ☐ 11.1 Update Order model
 - Remove vendor relationships
 - Add coupon, invoice relationships
 - Add order number generation
 - Add calculateTotals method
 - Add isPosOrder(), isOnlineOrder() helpers
 - *Requirements: 7.1, 7.4, 7.5, 8.4*
 - ☐ 11.2 Update OrderItem model

- Remove vendor relationship
 - Add productVariation relationship
 - *Requirements: 7.1*
- ☐ 11.3 Create Invoice model
 - Add order relationship
 - Add invoice number generation
 - *Requirements: 18.1, 18.5*
- []* 11.4 Write property test for order total calculation
 - **Property 16: Order Total Calculation**
 - **Validates: Requirements 7.5**
- []* 11.5 Write property test for order number uniqueness
 - **Property 17: Order Number Uniqueness**
 - **Validates: Requirements 7.4**
- ☐
- 12. Update Eloquent Models - Promotions and Coupons
 - ☐ 12.1 Update Promotion model
 - Add products, brands, categories relationships
 - Add scopeActive, isValidNow methods
 - Add calculateDiscount method
 - *Requirements: 9.1, 9.4, 9.5*
 - ☐ 12.2 Create Coupon model
 - Add products, brands, categories, usages relationships
 - Add isValid, canBeUsedBy methods
 - Add calculateDiscount method
 - *Requirements: 10.1, 10.3, 10.4, 10.5*
 - ☐ 12.3 Create CouponUsage model
 - Add coupon, user, order relationships
 - *Requirements: 10.6*
 - []* 12.4 Write property test for coupon single use per email
 - **Property 24: Coupon Single Use Per Email**
 - **Validates: Requirements 10.3**
 - []* 12.5 Write property test for promotion non-stacking
 - **Property 22: Promotion Non-Stacking**
 - **Validates: Requirements 9.3**
- ☐
- 13. Update Eloquent Models - Shipping and Inventory
 - ☐ 13.1 Update ShippingRate model
 - Add shippingClass, weightCostRules relationships
 - Add calculateCost method
 - *Requirements: 6.2, 6.4*
 - ☐ 13.2 Create InventoryLog model

- Add product, productVariation, createdBy relationships
 - *Requirements: 5.5*
 - []* 13.3 Write property test for inventory order relationship
 - **Property 10: Inventory Order Relationship**
 - **Validates: Requirements 5.3, 5.4, 8.7, 12.6**
- ☐
- 14. Update Eloquent Models - Reviews, Campaigns, CMS
 - ☐ 14.1 Update Review model
 - Remove shop relationship
 - Add helpfulUsers relationship
 - Add scopeApproved, scopePending
 - *Requirements: 13.1, 13.3*
 - ☐ 14.2 Create Campaign model
 - Add recipients relationship
 - Add scheduling and status methods
 - *Requirements: 11.1, 11.2, 11.5*
 - ☐ 14.3 Create CampaignRecipient model
 - Add campaign, user relationships
 - *Requirements: 11.3*
 - ☐ 14.4 Update StorePolicy model
 - Remove shop relationship
 - Add slug generation
 - *Requirements: 14.1*
 - ☐ 14.5 Create CmsPage model
 - Add slug generation, scopeActive
 - *Requirements: 14.2*
 - ☐ 14.6 Update Banner model (rename from PromotionalBanner)
 - Add scheduling scopes
 - *Requirements: 15.1, 15.3*
 - []* 14.7 Write property test for review rating range
 - **Property 31: Review Rating Range**
 - **Validates: Requirements 13.1**
- ☐
- 15. Checkpoint - Models Complete
 - Verify all model relationships work correctly
 - Ensure all tests pass, ask the user if questions arise.
- ☐
- 16. Implement Services - User and Authentication
 - ☐ 16.1 Create UserService

- Implement register, authenticate, updateProfile, deactivate
 - Implement sendVerificationEmail, resetPassword
 - *Requirements: 1.2, 1.3, 1.5, 1.6*
- []* 16.2 Write property test for authentication correctness
 - **Property 3: Authentication Correctness**
 - **Validates: Requirements 1.5**
- ☐
- 17. Implement Services - Catalog
 - ☐ 17.1 Create CatalogService
 - Implement createProduct, updateProduct, getProductWithVariations
 - Implement searchProducts, getProductsByCategory, getProductsByBrand
 - *Requirements: 3.1, 4.4*
 - ☐ 17.2 Create VariationService
 - Implement createVariation, updateStock, getAvailableOptions
 - *Requirements: 3.2, 3.3*
 - []* 17.3 Write property test for inactive product visibility
 - **Property 8: Inactive Product Visibility**
 - **Validates: Requirements 3.6**
- ☐
- 18. Implement Services - Inventory
 - ☐ 18.1 Create InventoryService
 - Implement checkAvailability, reserveStock, releaseStock
 - Implement adjustStock with logging
 - Implement getLowStockProducts
 - *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5*
 - []* 18.2 Write property test for stock status calculation
 - **Property 13: Stock Status Calculation**
 - **Validates: Requirements 5.6**
 - []* 18.3 Write property test for inventory adjustment audit
 - **Property 12: Inventory Adjustment Audit**
 - **Validates: Requirements 5.5**
- ☐
- 19. Implement Services - Shipping
 - ☐ 19.1 Create ShippingService
 - Implement calculateShippingCost with weight-based rules
 - Implement getAvailableMethods
 - Implement assignTrackingNumber
 - *Requirements: 6.1, 6.2, 6.4, 6.5*
 - []* 19.2 Write property test for shipping cost calculation

- **Property 14: Shipping Cost Calculation**
 - **Validates: Requirements 6.2, 6.4**
 - []* 19.3 Write property test for free shipping application
 - **Property 15: Free Shipping Application**
 - **Validates: Requirements 6.6**
- ☐
- 20. Implement Services - Promotions and Coupons
 - ☐ 20.1 Create PromotionService
 - Implement getActivePromotions, applyPromotion
 - Implement getBestPromotion (non-stacking logic)
 - *Requirements: 9.1, 9.2, 9.3, 9.4*
 - ☐ 20.2 Create CouponService
 - Implement validateCoupon, applyCoupon
 - Implement recordCouponUsage
 - Check once-per-email restriction
 - *Requirements: 10.3, 10.4, 10.5, 10.6*
 - []* 20.3 Write property test for coupon expiration validation
 - **Property 25: Coupon Expiration Validation**
 - **Validates: Requirements 10.5**
 - []* 20.4 Write property test for promotion date validity
 - **Property 23: Promotion Date Validity**
 - **Validates: Requirements 9.4**
- ☐
- 21. Implement Services - Orders
 - ☐ 21.1 Create OrderService
 - Implement createOrder for online checkout
 - Implement createPosOrder for in-store sales
 - Implement updateStatus, cancelOrder
 - Integrate inventory, promotion, coupon services
 - *Requirements: 7.1, 7.2, 7.6, 8.1, 8.2, 8.3, 8.4*
 - []* 21.2 Write property test for POS order marking
 - **Property 19: POS Order Marking**
 - **Validates: Requirements 8.4**
- ☐
- 22. Implement Services - Payments and Invoices
 - ☐ 22.1 Create PaymentService
 - Implement processPayment for SSL gateway
 - Implement recordManualPayment for POS
 - Implement handlePaymentCallback

- *Requirements: 17.1, 17.2, 17.3, 17.5*
- ☐ 22.2 Create InvoiceService
 - Implement generateInvoice with PDF
 - Implement sendInvoiceEmail
 - Implement regenerateInvoice
 - *Requirements: 18.1, 18.2, 18.3, 18.4*
- []* 22.3 Write property test for invoice number sequence
 - **Property 37: Invoice Number Sequence**
 - **Validates: Requirements 18.5**
- []* 22.4 Write property test for invoice data completeness
 - **Property 38: Invoice Data Completeness**
 - **Validates: Requirements 18.2**
- []* 22.5 Write property test for payment status order update
 - **Property 36: Payment Status Order Update**
 - **Validates: Requirements 17.3**
- ☐
- 23. Implement Services - Returns and Refunds
 - ☐ 23.1 Create ReturnService
 - Implement createReturnRequest, approveReturn, rejectReturn
 - Implement processReturn with inventory restoration
 - *Requirements: 12.2, 12.5, 12.6*
 - ☐ 23.2 Create RefundService
 - Implement createRefund, processRefund
 - Support partial refunds
 - *Requirements: 12.3, 12.4*
 - []* 23.3 Write property test for refund method default
 - **Property 29: Refund Method Default**
 - **Validates: Requirements 12.3**
- ☐
- 24. Implement Services - Reviews
 - ☐ 24.1 Create ReviewService
 - Implement createReview with purchase verification
 - Implement approveReview, rejectReview
 - Implement markHelpful, calculateAverageRating
 - *Requirements: 13.1, 13.2, 13.3, 13.4, 13.5*
 - []* 24.2 Write property test for review purchase verification
 - **Property 32: Review Purchase Verification**
 - **Validates: Requirements 13.2**
 - []* 24.3 Write property test for average rating calculation
 - **Property 33: Average Rating Calculation**
 - **Validates: Requirements 13.4**

- ☐

25. Implement Services - Campaigns

- ☐ 25.1 Create CampaignService
 - Implement createCampaign, getTargetCustomers
 - Implement sendCampaign with queue
 - Implement trackEmailOpen, getCampaignStats
 - *Requirements: 11.1, 11.2, 11.3, 11.4, 11.5*
- []* 25.2 Write property test for campaign recipient targeting
 - **Property 27: Campaign Recipient Targeting**
 - **Validates: Requirements 11.2**

- ☐

26. Checkpoint - Services Complete

- Run all service tests
- Ensure all tests pass, ask the user if questions arise.

- ☐

27. Implement Controllers - Admin

- ☐ 27.1 Create Admin UserController
 - List, view, edit, deactivate customers
 - *Requirements: 1.4*
- ☐ 27.2 Create Admin CategoryController
 - CRUD operations with hierarchy support
 - *Requirements: 2.1, 2.2, 2.3, 2.4*
- ☐ 27.3 Create Admin ProductController
 - CRUD with variations, images, SEO
 - *Requirements: 3.1, 3.2, 3.4, 3.7*
- ☐ 27.4 Create Admin BrandController and ModelController
 - CRUD operations
 - *Requirements: 4.1, 4.3*
- ☐ 27.5 Create Admin OrderController
 - List, view, update status, cancel
 - *Requirements: 7.6*
- ☐ 27.6 Create Admin POSController
 - Create POS orders, apply discounts
 - *Requirements: 8.1, 8.2, 8.3*
- ☐ 27.7 Create Admin PromotionController
 - CRUD with targeting options
 - *Requirements: 9.1, 9.4, 9.5*
- ☐ 27.8 Create Admin CouponController
 - CRUD with usage tracking
 - *Requirements: 10.1, 10.2*

- ☐ 27.9 Create Admin ShippingController
 - Manage rates, classes, weight rules
 - *Requirements: 6.1, 6.3, 6.4*
- ☐ 27.10 Create Admin InventoryController
 - View stock, adjust inventory, view logs
 - *Requirements: 5.1, 5.2, 5.5*
- ☐ 27.11 Create Admin ReturnController and RefundController
 - Process returns and refunds
 - *Requirements: 12.3, 12.5*
- ☐ 27.12 Create Admin ReviewController
 - Approve, reject, respond to reviews
 - *Requirements: 13.3*
- ☐ 27.13 Create Admin CampaignController
 - Create, schedule, send campaigns
 - *Requirements: 11.1, 11.2, 11.5*
- ☐ 27.14 Create Admin ContentController
 - Manage policies, CMS pages, banners
 - *Requirements: 14.1, 14.2, 15.1, 15.2*
- ☐ 27.15 Create Admin ReportController
 - Sales, product, customer, inventory reports
 - *Requirements: 16.1, 16.2, 16.3, 16.4, 16.5*
- ☐

28. Implement Controllers - Customer/Public

- ☐ 28.1 Create AuthController
 - Register, login, logout, password reset
 - *Requirements: 1.2, 1.3, 1.5, 1.6*
- ☐ 28.2 Create CatalogController
 - Browse categories, products, search, filter
 - *Requirements: 2.5, 3.6, 4.4*
- ☐ 28.3 Create CartController
 - Add, update, remove items
 - Apply coupons
 - *Requirements: 10.4*
- ☐ 28.4 Create CheckoutController
 - Process checkout, calculate shipping
 - *Requirements: 7.1, 6.2*
- ☐ 28.5 Create PaymentController
 - Handle payment gateway callbacks
 - *Requirements: 17.1, 17.3, 17.5*
- ☐ 28.6 Create OrderController (Customer)
 - View order history, track orders
 - *Requirements: 6.5*

- ☐ 28.7 Create ReviewController (Customer)
 - Submit reviews, mark helpful
 - *Requirements: 13.1, 13.2, 13.5*
- ☐ 28.8 Create ReturnController (Customer)
 - Request returns
 - *Requirements: 12.5*
- ☐ 28.9 Create PageController
 - Display policies, CMS pages
 - *Requirements: 14.1, 14.2*
- ☐
- 29. Implement Email Notifications
 - ☐ 29.1 Create order notification emails
 - Order confirmation, status updates
 - *Requirements: 7.3*
 - ☐ 29.2 Create invoice email with PDF attachment
 - *Requirements: 18.3*
 - ☐ 29.3 Create campaign email templates
 - *Requirements: 11.4*
 - ☐ 29.4 Create user verification and password reset emails
 - *Requirements: 1.3, 1.6*
- ☐
- 30. Implement Event Listeners
 - ☐ 30.1 Create OrderPlaced listener
 - Decrement inventory, send confirmation
 - *Requirements: 5.3, 7.3*
 - ☐ 30.2 Create OrderStatusChanged listener
 - Send status update email
 - *Requirements: 7.3*
 - ☐ 30.3 Create PaymentCompleted listener
 - Update order status, generate invoice
 - *Requirements: 17.3, 18.1*
 - ☐ 30.4 Create LowStockAlert listener
 - Notify admin when threshold reached
 - *Requirements: 5.2*
 - ☐ 30.5 Create ReturnCompleted listener
 - Restore inventory
 - *Requirements: 12.6*
- ☐
- 31. Final Checkpoint

- Run complete test suite
- Verify all migrations run cleanly
- Test critical user flows
- Ensure all tests pass, ask the user if questions arise.

Notes

- Tasks marked with * are optional and can be skipped for faster MVP
- Each task references specific requirements for traceability
- Checkpoints ensure incremental validation
- Property tests validate universal correctness properties
- Unit tests validate specific examples and edge cases